

Practical 9

Implement the following JPA applications.

- Develop a simple Inventory Application Using JPA.
- Develop a Guestbook Application Using JPA.
- Create simple JPA application to store and retrieve Book details.

What is JPA?

JPA (Java Persistence API) is a Java specification used to manage and persist objects in a relational database.

JPA allows Java objects to be mapped to database tables without writing SQL queries for every database operation.

What is ORM?

ORM (Object-Relational Mapping) is a technique that maps Java objects to relational database tables.

For example:

Java Class		Database Table
Student	→	student
Department	→	department

The attributes of a Java class are mapped to columns of a database table.

JPA Architecture

Java Application



Entity



JPA / ORM



Persistence Provider



Database

Steps

Step 1: Create a Java Project

1. Open **NetBeans IDE**.
2. Select **File → New Project**.
3. Select **Java with Maven → Java Application**.
4. Give the project name:

InventoryJPA

5. Click **Finish**.

Step 2: Add JPA Dependencies pom.xml

```
<dependency>
```

```
    <groupId>org.hibernate.orm</groupId>
```

```
    <artifactId>hibernate-core</artifactId>
```

```
    <version>6.6.5.Final</version>
```

```
</dependency>
```

```
<dependency>
```

```
    <groupId>com.mysql</groupId>
```

```
    <artifactId>mysql-connector-j</artifactId>
```

```
    <version>9.1.0</version>
```

```
</dependency>
```

Step 3: Create a Package

Right-click **Source Packages**.

Select:

New → Java Package

Enter:

com.inv

Click **Finish**.

Step 4: Create the Product Entity

Right-click the com.inv package.

Select:

New → Java Class

Enter:

Product

Click **Finish**.

Step 5: Write the Product Class

Step 6: Create persistence.xml

Right-click on META-INF -New-Other-Persistence Unit

Create the following folder structure:

src

└─ main

└─ resources

└─ META-INF

└─ persistence.xml

The file name must be:

persistence.xml

Step 7: Configure Persistence Unit

Important

The persistence unit name is:

InventoryPU

This same name must be used in the Java program:

`Persistence.createEntityManagerFactory("InventoryPU");`

Step 9: Create InventoryApp Class

Right-click the com.inv package.

Select:

New → Java Class

Enter:

InventoryApp

Click Finish.

Step 10: Write the InventoryApp Program

Product.java

```
package com.inv;
```

```
import jakarta.persistence.Entity;
```

```
import jakarta.persistence.Id;
```

```
@Entity
```

```
public class Product {
```

```
    @Id
```

```
    private int id;
```

```
    private String name;
```

```
    private int quantity;
```

```
    private double price;
```

```
    public int getId() {
```

```
        return id;
```

```
    }
```

```
    public void setId(int id) {
```

```
        this.id = id;
```

```
    }
```

```
public String getName() {  
    return name;  
}
```

```
public void setName(String name) {  
    this.name = name;  
}
```

```
public int getQuantity() {  
    return quantity;  
}
```

```
public void setQuantity(int quantity) {  
    this.quantity = quantity;  
}
```

```
public double getPrice() {  
    return price;  
}
```

```
public void setPrice(double price) {  
    this.price = price;  
}
```

```
}
```

InventoryApp.java

```
package com.inv;

import jakarta.persistence.EntityManager;
import jakarta.persistence.EntityManagerFactory;
import jakarta.persistence.Persistence;

public class InventoryApp {
    public static void main(String[] args) {

        // Create EntityManagerFactory
        EntityManagerFactory emf =
            Persistence.createEntityManagerFactory("InventoryPU");

        // Create EntityManager
        EntityManager em = emf.createEntityManager();

        // Create Product object
        Product p = new Product();

        p.setId(1);
        p.setName("Pen");
        p.setQuantity(50);
        p.setPrice(10.0);

        // Store product in database
        em.getTransaction().begin();

        em.persist(p);
```

```
em.getTransaction().commit();
```

```
System.out.println("Product added successfully!");
```

```
// Retrieve product
```

```
Product product = em.find(Product.class, 1);
```

```
// Display product details
```

```
System.out.println("Product ID: " + product.getId());
```

```
System.out.println("Product Name: " + product.getName());
```

```
System.out.println("Quantity: " + product.getQuantity());
```

```
System.out.println("Price: " + product.getPrice());
```

```
// Close
```

```
em.close();
```

```
emf.close();
```

```
}
```

```
}
```

persistence.xml

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<persistence xmlns="https://jakarta.ee/xml/ns/persistence"
    version="3.1">
```

```
<persistence-unit name="InventoryPU">
```

```
<provider>
```

```
    org.hibernate.jpa.HibernatePersistenceProvider
```

```
</provider>
```

```
<class>com.inv.Product</class>
```

```
<properties>
```

```
<property name="jakarta.persistence.jdbc.driver"
    value="com.mysql.cj.jdbc.Driver"/>
```

```
<property name="jakarta.persistence.jdbc.url"
    value="jdbc:mysql://localhost:3306/mysql"/>
```

```
<property name="jakarta.persistence.jdbc.user"
    value="root"/>
```

```
<property name="jakarta.persistence.jdbc.password"
    value=""/>
```

```
<property name="hibernate.hbm2ddl.auto"
    value="update"/>
```

`</properties>`

`</persistence-unit>`

`</persistence>`

****Remember to add class in persistence.xml at last****